

PRONTUÁRIO: _____ **Data:** __/__/____ **Horário:** 3h00

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código implementado por você anteriormente. ✓ Não coloque seu nome no projeto. ✓ Atribui-se nota zero à prova em desacordo com os itens acima.	✓ A prova deve ser nomeada da seguinte forma: PRONTUARIO_P_SIMULADA, com o "SC". ✓ Não envie apenas as classes, mas todo o projeto. ✓ Envie o projeto como zip (ou apenas os .java, se for só treino pessoal).
--	--

Considere o contexto a seguir:

Você vai implementar o núcleo de um sistema de controle de estacionamento. Um colega já implementou a classe Veiculo (placa, modelo). Você deve implementar as classes Ticket (representa a permanência de um veículo) e Estacionamento (gerencia as vagas e os tickets), conforme o diagrama a seguir:

Estacionamento	Ticket
- vagasTotais: int - tickets: Ticket[] - valorHora: double	- codigo: String - veiculo: Veiculo - entrada: LocalDateTime - saida: LocalDateTime
+ Estacionamento(vagasTotais: int, valorHora: double) + registrarEntrada(v: Veiculo): Ticket + registrarSaida(placa: String): double + vagasDisponiveis(): int + veiculosNoPatio(): Veiculo[]	+ Ticket(codigo: String, veiculo: Veiculo) + finalizar(): void + estaAberto(): boolean + tempoPermanencia(): Duration + valorCobrado(valorHora: double): double

Relacionamentos: Estacionamento possui de 0 a N objetos Ticket (composição, array, tamanho máximo = vagasTotais). Cada Ticket está associado a exatamente 1 Veiculo (relacionamento unidirecional; Veiculo não conhece Ticket nem Estacionamento).

Regras de negócio:

- A placa de um Veiculo segue o padrão Mercosul: 3 letras maiúsculas, 1 dígito, 1 letra maiúscula, 2 dígitos (ex.: "ABC1D23"). Como o construtor de Veiculo já valida isso (feito pelo colega), você deve apenas validar esse mesmo padrão, via regex, no método registrarEntrada() do Estacionamento antes de aceitar o veículo — trate como erro (lance exceção) qualquer placa fora do padrão que eventualmente chegue malformada.
- entrada de um Ticket é sempre o momento da criação (LocalDateTime.now()).
- registrarEntrada(v) só aceita o veículo se houver vaga disponível (vagasDisponiveis() > 0); o código do ticket deve ser gerado concatenando a placa do veículo com um timestamp/sequencial simples, garantindo que seja único.
- finalizar() registra saida como o momento atual.
- tempoPermanencia() retorna a Duration entre entrada e saida (ou entre entrada e o momento atual, se o ticket ainda estiver aberto) — utilize a API Date/Time (Duration.between), nunca cálculo manual.
- valorCobrado(valorHora) cobra proporcionalmente às horas de permanência, sempre arredondando a fração de hora para cima (ex.: 1h05min conta como 2 horas). Dica: use os métodos de Duration (toMinutes(), etc.) para o arredondamento.
- registrarSaida(placa), no Estacionamento, localiza o ticket aberto correspondente à placa, chama finalizar(), calcula e retorna o valor cobrado (usando o valorHora do estacionamento).
- veiculosNoPatio() retorna os veículos cujo ticket ainda está aberto (não finalizado).
- vagasDisponiveis() = vagasTotais menos a quantidade de tickets atualmente abertos.

Execute as atividades a seguir para a implementação do exercício em Java. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução.

#	Descrição	Pont.
1	Crie as classes do modelo com os modificadores de acesso adequados ao encapsulamento.	0,5pt
2	Estruture a associação de composição entre Estacionamento e Ticket usando array de tamanho vagasTotais.	0,5pt
3	Implemente o construtor de Ticket e o método finalizar()/estaAberto().	1,0pt
4	Implemente tempoPermanencia() usando Duration, cobrindo corretamente os dois casos (ticket aberto e fechado).	1,5pt
5	Implemente valorCobrado(valorHora), com o arredondamento de hora para cima calculado a partir da Duration (sem cálculo manual de milissegundos).	1,5pt
6	Implemente registrarEntrada(v) no Estacionamento, validando a placa via regex, verificando vaga disponível e gerando um código de ticket único.	1,5pt
7	Implemente registrarSaida(placa), localizando o ticket certo, finalizando-o e retornando o valor cobrado.	1,0pt
8	Implemente vagasDisponiveis() e veiculosNoPatio().	1,0pt
9	Crie uma classe Principal com um método main() que crie um Estacionamento com poucas vagas (ex.: 2), tente registrar mais entradas do que vagas disponíveis (tratando o erro), registre ao menos uma saída e imprima o valor cobrado e as vagas disponíveis ao final.	1,0pt